

บทที่ 9 : การพัฒนาเว็บ แอปพลิเคชันส่วนฟรอนต์เอนด์

1. เริ่มต้นสร้าง Front-End Web
2. สร้างหน้า Register
3. สร้างหน้า Login
4. สร้างหน้า User Profile
5. แบบฝึกหัด



1. เริ่มต้นสร้าง Front-End Web

1. เริ่มสร้างโปรเจกต์
npx create-next-app@latest **my-app** --typescript
แล้วเลือกตอบ “Yes” หรือ “No” ดังนี้

ระบุเครื่องหมายจุด . แทนชื่อโปรเจกต์ หากไม่ต้องการให้สร้าง folder ใหม่

- ✓ Would you like to use ESLint? ... No / Yes
- ✓ Would you like to use Tailwind CSS? ... No / Yes
- ✓ Would you like your code inside a `src/` directory? ... No / Yes
- ✓ Would you like to use App Router? (recommended) ... No / Yes
- ✓ Would you like to use Turbopack for `next dev`? ... No / Yes
- ✓ Would you like to customize the import alias (`@/\*` by default)? ... No / Yes

2. รัน Web App (หากมีการตั้งชื่อโปรเจกต์ให้ใช้คำสั่ง cd **my-app**)
npm run dev

3. ทดสอบเรียก url จาก web browser
http://localhost:3000



localhost:3000



Enter passphrase

NEXT.js

1. Get started by editing `src/app/page.tsx`.
2. Save and see your changes instantly.

▲ Deploy now

Read our docs



Learn



Examples



Go to nextjs.org →



4. เขียนโค้ดไว้ในไฟล์ /src/app/customers/page.tsx

```
1 // 1. Importing Dependencies
2 // - Import libraries, components, styles, etc.
3 "use client"; // ต้องใช้เพราะใช้ useState และ window, localStorage
4 import React from 'react';
5
6 // 2. Creating and Exporting a Component
7 // - Define the component function/class and export it
8 export default function Home() {
9
10 // 2.1 Defining Variables, States, and Handlers
11 // - ตั้งค่า useState, useEffect, event handlers ฯลฯ
12 const [name, setName] = React.useState('');
13 const handleInputChange = (e: React.ChangeEvent<HTMLInputElement>) => {
14   setName(e.target.value);
15 };
16
17 // 2.2 Returning UI Output
18 // - ทำการ return JSX เพื่อแสดงผล UI
19 return (
20   <div className="flex flex-col items-center
21     justify-center min-h-screen bg-gray-100">
22     <h1 className="text-3xl font-bold mb-4">Hello, {name || "World"}!</h1>
23     <input
24       type="text"
25       placeholder="Enter your name"
26       className="border p-2 rounded"
27       value={name}
28       onChange={handleInputChange}
29     />
30   </div>
31 );
32 }
```

ตารางที่ 1-1 : อธิบาย HTML Tags

HTML Tag	คำอธิบาย
<div>	Container หลักสำหรับจัด layout (wrap ทุกอย่างไว้ด้วยกัน)
<h1>	หัวข้อหลักของหน้า ใช้สำหรับแสดงข้อความใหญ่ เช่น ชื่อหน้า
<input>	กล่องรับค่าจากผู้ใช้ (ในที่นี้คือ input ชื่อ)

ตารางที่ 1-2 : อธิบาย Tailwind CSS Classes

Tailwind Class	คำอธิบาย
flex	กำหนดให้ใช้ Flexbox layout
flex-col	จัดเรียงลูกภายใน <div> ให้เป็นแนวตั้ง (column)
items-center	จัดตำแหน่งแนวนอน (horizontal) ของ items ให้อยู่ตรงกลาง
justify-center	จัดตำแหน่งแนวตั้ง (vertical) ของ items ให้อยู่ตรงกลาง
min-h-screen	ความสูงขั้นต่ำของ container เท่ากับความสูงของหน้าจอ
bg-gray-100	กำหนดพื้นหลังให้เป็นสีเทาอ่อน (ระดับ 100)
text-3xl	ขนาดตัวอักษรขนาดใหญ่มาก (3xl)
font-bold	กำหนดตัวอักษรให้หนา (bold)
mb-4	margin-bottom = 1rem (16px)
border	เพิ่มเส้นขอบรอบ <input>
p-2	padding รอบ input 0.5rem (8px)
rounded	มุมของ input โค้งมน (border-radius)

5. ทดสอบเรียก url จาก web browser

`http://localhost:3000/customers`

Hello, World!

Enter your name

2. สร้างหน้า Register

1. ทำการติดตั้ง Axios เพื่อให้สามารถเรียกใช้ API ได้ โดยใช้คำสั่ง ดังนี้

npm install axios

2. จากนั้นให้เขียนโค้ดไว้ใน `/src/app/customers/register/page.tsx`

```
1  // 1. Importing Dependencies
2  "use client"; // ต้องใช้เพราะใช้ useState และ window, localStorage
3  import { useState, FormEvent } from "react";
4  import axios from "axios";
5
6  // 2. Creating and Exporting a Component
7  export default function Register() {
8      // 2.1 Defining Variables, States, and Handlers
9
10     // สร้างตัวแปร state สำหรับเก็บ username, password, firstName และ lastName
11     const [username, setUsername] = useState<string>("");
12     const [password, setPassword] = useState<string>("");
13     const [firstName, setFirstName] = useState<string>("");
14     const [lastName, setLastName] = useState<string>("");
```

```

15
16 // สร้างฟังก์ชันสำหรับการจัดการการ submit form ไปยัง API
17 // โดยใช้ async/await เพื่อจัดการกับการเรียก API แบบ asynchronous
18 const handleSubmit = async (e: FormEvent<HTMLFormElement>) => {
19     e.preventDefault(); // ป้องกัน reload หน้า
20
21     try {
22         // ใช้ axios เพื่อส่ง POST request ไปยัง API
23         const response = await axios.post("http://localhost:4000/api/register", {
24             username,
25             password,
26             firstName,
27             lastName,
28         });
29
30         const result = response.data; // รับข้อมูลจาก API
31         alert(result.message); // แสดงข้อความจาก API
32
33         // ถ้าสมัครสมาชิกสำเร็จจะ redirect ไปหน้าแรก
34         if (result.status === true) {
35             window.location.href = "/";
36         }
37     } catch (err) {
38         console.error("Registration error:", err); // แสดงข้อผิดพลาดใน console
39         alert("Registration failed."); // แสดงข้อความเมื่อสมัครสมาชิกไม่สำเร็จ
40     }
41 };

```

async/await เป็นการกำหนดให้ **asynchronous function** ทำงานเสมือนเป็นแบบลำดับขั้นตอน (**synchronous**) – ในกรณีนี้หากไม่ใช้ **async/await** บรรทัดที่ 30 อาจจะเริ่มทำงานในขณะที่บรรทัดที่ 23 ยังทำงานไม่เสร็จ

```

42
43 // 2.2 Returning UI Output
44 return (
45   <div className="min-h-screen flex items-center justify-center bg-gray-100 p-4">
46     <form
47       onSubmit={handleSubmit}
48       className="bg-white shadow-md rounded-lg p-6 w-full max-w-md space-y-4"
49     >
50       <h1 className="text-2xl font-semibold text-center text-gray-800">
51         ระบบซื้อขายออนไลน์
52       </h1>
53
54       <input
55         type="text"
56         placeholder="Username"
57         value={username}
58         onChange={(e) => setUsername(e.target.value)}
59         className="w-full px-4 py-2 border rounded-md focus:outline-none focus:ring-2 focus:ring-blue-500"
60       />
61
62       <input
63         type="password"
64         placeholder="Password"
65         value={password}
66         onChange={(e) => setPassword(e.target.value)}
67         className="w-full px-4 py-2 border rounded-md focus:outline-none focus:ring-2 focus:ring-blue-500"
68       />

```



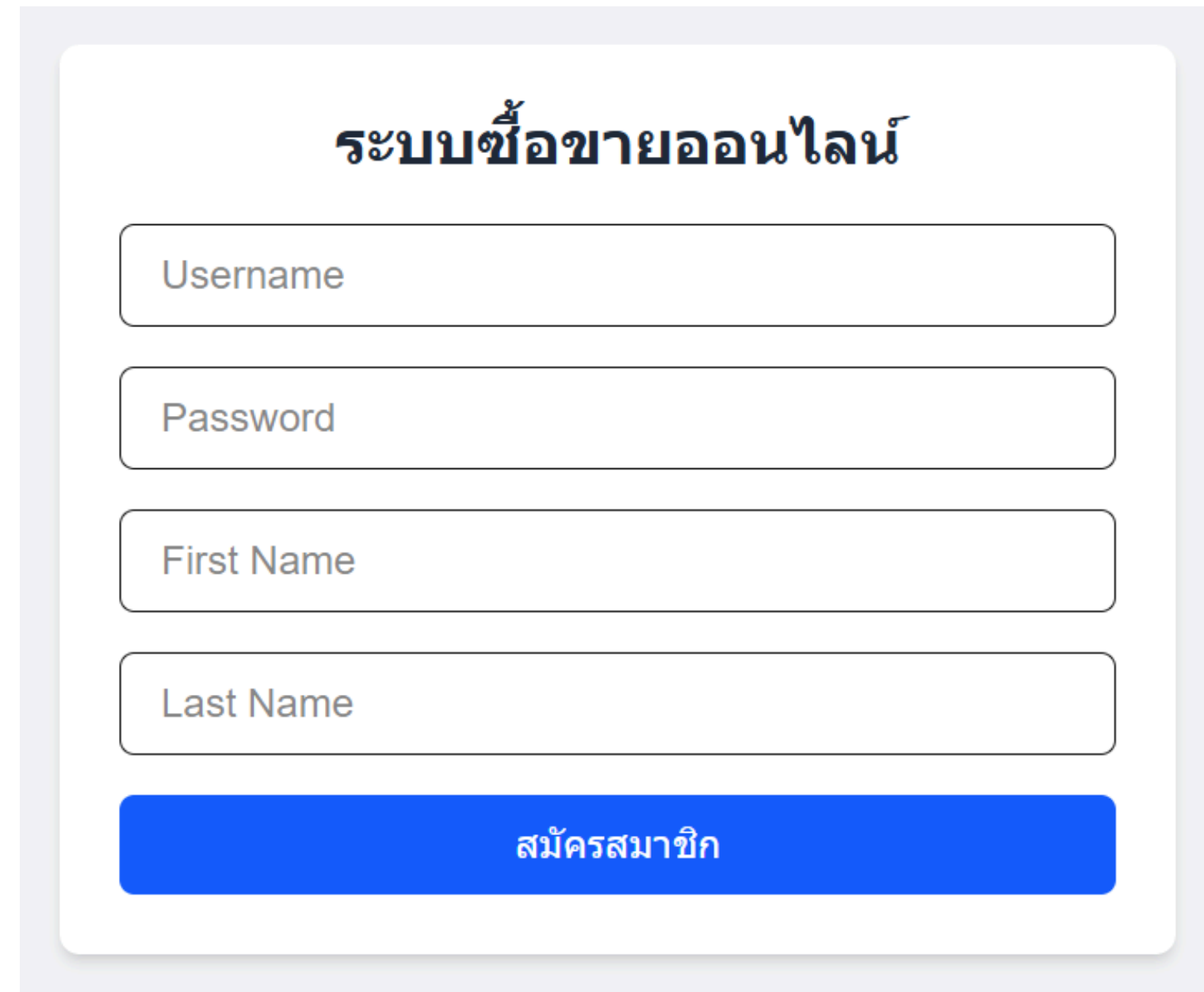
```

69
70     <input
71       type="text"
72       placeholder="First Name"
73       value={firstName}
74       onChange={(e) => setFirstName(e.target.value)}
75       className="w-full px-4 py-2 border rounded-md focus:outline-none focus:ring-2 focus:ring-blue-500"
76     />
77
78     <input
79       type="text"
80       placeholder="Last Name"
81       value={lastName}
82       onChange={(e) => setLastName(e.target.value)}
83       className="w-full px-4 py-2 border rounded-md focus:outline-none focus:ring-2 focus:ring-blue-500"
84     />
85
86     <button
87       type="submit"
88       className="w-full bg-blue-600 hover:bg-blue-700 text-white py-2 px-4 rounded-md transition"
89     >
90       สมัครสมาชิก
91     </button>
92   </form>
93 </div>
94 );
95 }

```

3. ทดสอบเรียก url จาก web browser

<http://localhost:3000/customers/register>



ระบบซื้อขายออนไลน์

Username

Password

First Name

Last Name

สมัครสมาชิก

ทำการ copy โค้ดเดิมแล้วส่งให้ Claude จากนั้นให้ prompt ดังนี้
“change this to modern web design (Responsive) with TailwindCSS”

The image shows a login form for an online trading system. At the top, there is a blue square icon with a white person silhouette. Below it, the title 'ระบบซื้อขายออนไลน์' (Online Trading System) is displayed in bold black text, followed by the subtitle 'สร้างบัญชีใหม่เพื่อเริ่มต้นใช้งาน' (Create a new account to get started) in a smaller font. The form consists of four input fields: 'Username' with a person icon, 'Password' with a lock icon and a toggle eye icon, 'First Name' with a person icon, and 'Last Name' with a person icon. Below these fields is a large blue button with the text 'สมัครสมาชิก' (Sign Up). At the bottom, there is a link that says 'มีบัญชีแล้ว? เข้าสู่ระบบ' (Already have an account? Log in).

3. สร้างหน้า Login

1. เขียนโค้ดไว้ใน /src/app/customers/login/page.tsx

```
1  // 1. Importing Dependencies
2  "use client"; // ต้องใช้เพราะใช้ useState และ window, localStorage
3  import { useState, FormEvent } from "react";
4  import axios from "axios";
5
6  // 2. Creating and Exporting a Component
7  export default function Login() {
8    // 2.1 Defining Variables, States, and Handlers
9
10   // สร้างตัวแปร state สำหรับเก็บ username และ password
11   const [username, setUsername] = useState<string>("");
12   const [password, setPassword] = useState<string>("");
```



```
13
14 // สร้างฟังก์ชันสำหรับการจัดการการ submit form ไปยัง API
15 // โดยใช้ async/await เพื่อจัดการกับการเรียก API แบบ asynchronous
16 const handleSubmit = async (e: FormEvent<HTMLFormElement>) => {
17     e.preventDefault(); // ป้องกัน reload หน้า
18
19     try {
20         // ใช้ axios เพื่อส่ง POST request ไปยัง API
21         const response = await axios.post("http://localhost:4000/api/login", {
22             username,
23             password
24         });
25
26         const result = response.data; // รับข้อมูลจาก API
27         alert(result.message); // แสดงข้อความจาก API
28
29         // ถ้าเข้าสู่ระบบสำเร็จ จะเก็บ token และ redirect ไปหน้าแรก
30         if (result.status === true) {
31             localStorage.setItem("token", result.token);
32             window.location.href = "/";
33         }
34     } catch (err) {
35         console.error("Login error:", err); // แสดงข้อผิดพลาดใน console
36         alert("Login failed."); // แสดงข้อความเมื่อเข้าสู่ระบบไม่สำเร็จ
37     }
38 };
```

```

39
40 // 2.2 Returning UI Output
41 return (
42   <form onSubmit={handleSubmit} className="max-w-md mx-auto mt-10 p-4 bg-white shadow rounded">
43     <h2 className="text-xl font-bold mb-4">เข้าสู่ระบบ</h2>
44
45     <input
46       type="text"
47       placeholder="ชื่อผู้ใช้"
48       value={username}
49       onChange={(e) => setUsername(e.target.value)}
50       className="w-full p-2 border border-gray-300 rounded mb-2"
51     />
52
53     <input
54       type="password"
55       placeholder="รหัสผ่าน"
56       value={password}
57       onChange={(e) => setPassword(e.target.value)}
58       className="w-full p-2 border border-gray-300 rounded mb-4"
59     />
60
61     <button
62       type="submit"
63       className="w-full bg-blue-500 text-white p-2 rounded hover:bg-blue-600"
64     >
65       เข้าสู่ระบบ
66     </button>
67   </form>
68 );
69 }

```

2. ทดสอบเรียก url จาก web browser

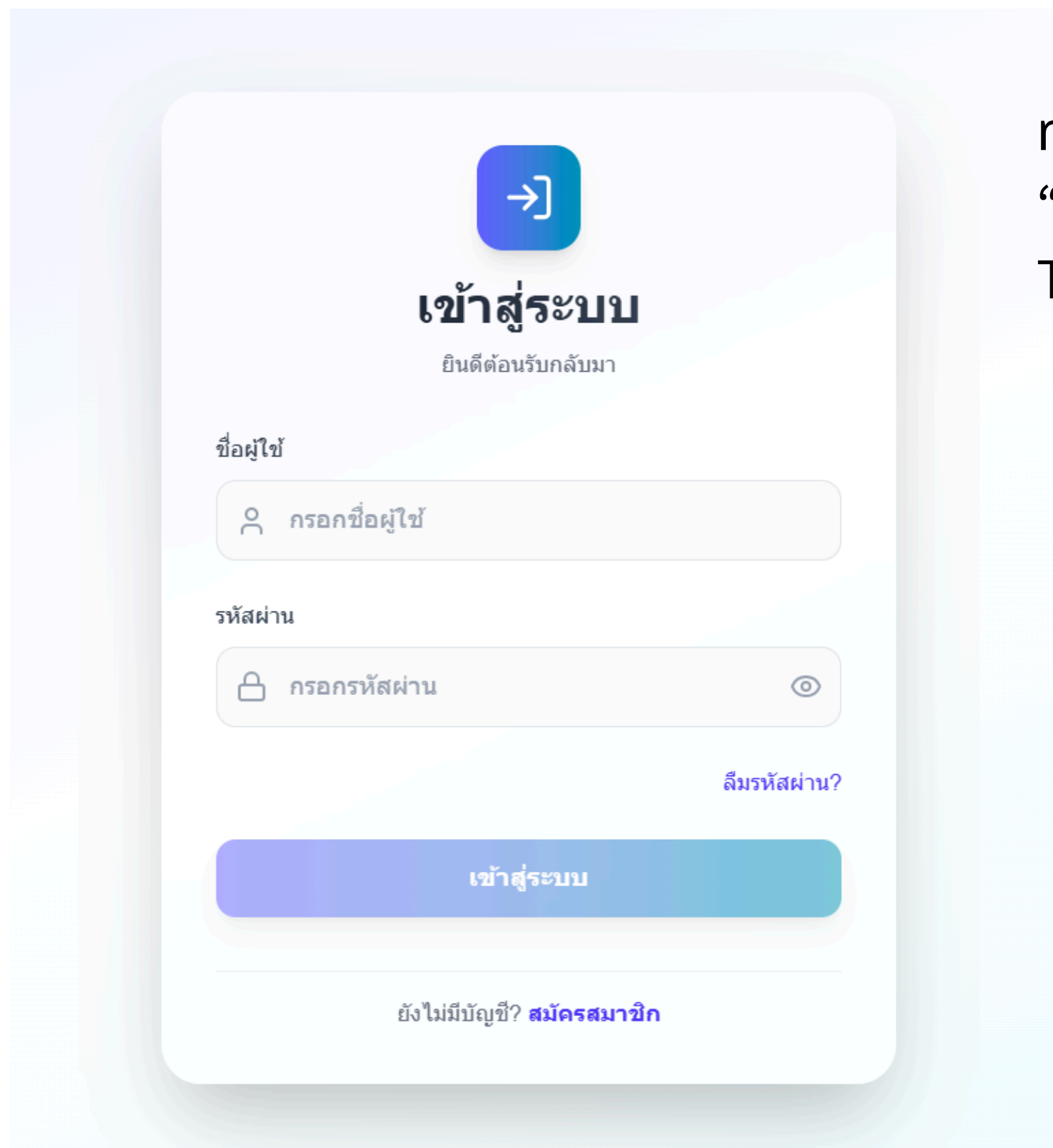
`http://localhost:3000/customers/login`

เข้าสู่ระบบ

ชื่อผู้ใช้

รหัสผ่าน

เข้าสู่ระบบ



ทำการ copy โค้ดเดิมแล้วส่งให้ Claude จากนั้นให้ prompt ดังนี้
“change this to modern web design (Responsive) with TailwindCSS”

4. สร้างหน้า User Profile

1. ทำการติดตั้ง lucide-react เพื่อใช้ Icons โดยใช้คำสั่ง ดังนี้ **npm install lucide-react**
2. เขียนโค้ดไว้ใน /src/app/customers/profile/[id]/page.tsx

```
1  // 1. Importing Dependencies
2  "use client";
3  import { useEffect, useState } from "react";
4  import { useParams } from "next/navigation";
5  import axios from "axios";
6  import { User } from "lucide-react";
7
8  // 2. Creating and Exporting a Component
9  // สร้าง Interface สำหรับข้อมูลลูกค้า
10 interface Customer {
11     custID: number;
12     username: string;
13     firstName: string;
14     lastName: string;
15     email: string;
16     mobilePhone: string;
17     birthdate: string;
18     gender: number;
19     address: string;
20     imageFile: string;
21 }
```



```

22
23 export default function UserProfile() {
24     // 2.1 Defining Variables, States, and Handlers
25
26     const { id } = useParams(); // รับ custID จาก URL
27     const [customer, setCustomer] = useState<Customer | null>(null);
28
29     //เรียก API ด้วย token ที่เก็บไว้ใน localStorage
30     useEffect(() => {
31         const fetchProfile = async () => {
32             try {
33                 const token = localStorage.getItem("token");
34
35                 if (!token) {
36                     alert("กรุณาเข้าสู่ระบบก่อน");
37                     window.location.href = "/login"; // redirect ไปหน้า login
38                     return;
39                 }
40
41                 const response = await axios.get(`http://localhost:4000/api/profile/${id}`, {
42                     headers: {
43                         Authorization: `Bearer ${token}`, // แนบ token
44                     },
45                 });
46
47                 setCustomer(response.data);
48             } catch (error) {
49                 console.error("Error fetching profile:", error);
50                 alert("ไม่สามารถโหลดข้อมูลโปรไฟล์ได้");
51             }
52         };
53
54         fetchProfile();
55     }, []);

```

```

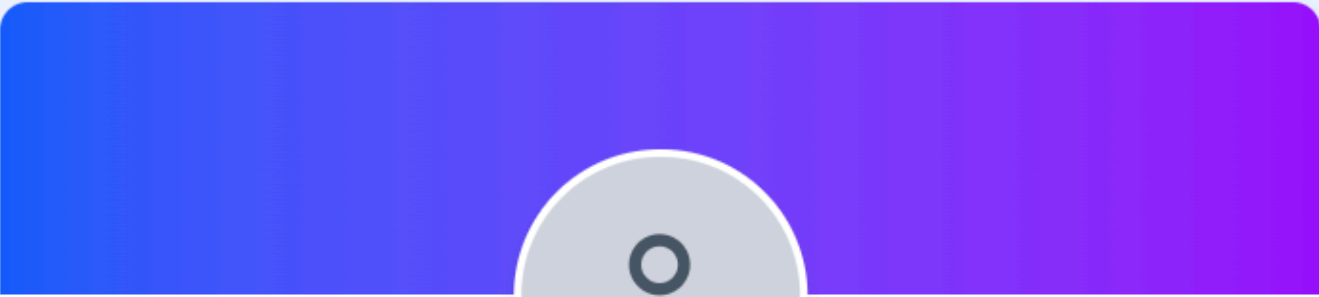
56
57 if (!customer) {// แสดง Loading ถ้ายังไม่มีข้อมูล
58 | return <div className="text-center py-10 text-gray-500">กำลังโหลดข้อมูล...</div>;
59 }
60
61 // 2.2 Returning UI Output
62 return (
63 | <div className="min-h-screen bg-gray-100 flex items-center justify-center p-4">
64 | | <div className="bg-white p-6 rounded-lg shadow-md w-full max-w-xl">
65 | | | <h2 className="text-2xl font-bold mb-4 text-center text-blue-800">โปรไฟล์ผู้ใช้</h2>
66 | | |
67 | | | <div className="flex flex-col items-center space-y-4">
68 | | | | <User className="w-32 h-32 object-cover rounded-full border border-gray-300"/>
69 | | | |
70 | | | | <div className="text-center">
71 | | | | | <h3 className="text-xl font-semibold">
72 | | | | | | {customer.firstName} {customer.lastName}
73 | | | | | </h3>
74 | | | | | <p className="text-gray-500">@{customer.username}</p>
75 | | | | </div>
76 | | | </div>
77 | | |
78 | | | <div className="mt-6 space-y-2 text-gray-700">
79 | | | | <p><strong>อีเมล:</strong> {customer.email || "-"}</p>
80 | | | | <p><strong>เบอร์โทร:</strong> {customer.mobilePhone || "-"}</p>
81 | | | | <p><strong>วันเกิด:</strong> {customer.birthdate || "-"}</p>
82 | | | | <p><strong>เพศ:</strong> {customer.gender === 0 ? "ชาย" : "หญิง"}</p>
83 | | | | <p><strong>ที่อยู่:</strong> {customer.address || "-"}</p>
84 | | | </div>
85 | | </div>
86 | </div>
87 | );
88 }

```

3. ทดสอบเรียก url จาก web browser

<http://localhost:3000/customers/profile/รหัสลูกค้า>





@somchai

 **เบอร์โทร: -**

 **วันเกิด: -**

⊖ เพศ: หญิง

 **ที่อยู่: -**

ทำการ copy โค้ดเดิมแล้วส่งให้ Gemini จากนั้นให้ prompt ดังนี้
“change this to modern web design (Responsive) with TailwindCSS”

5. แบบฝึกหัด

1. อธิบายว่า Next.js คืออะไร และมีข้อดีกว่า React.js อย่างไร
2. อธิบายความแตกต่างระหว่าง Client Component กับ Server Component พร้อมยกตัวอย่างสถานการณ์ที่ควรใช้แต่ละประเภท
3. เปรียบเทียบ Rendering ทั้ง 4 รูปแบบต่อไปนี้
 - CSR (Client-Side Rendering)
 - SSR (Server-Side Rendering)
 - SSG (Static Site Generation)
 - ISR (Incremental Static Regeneration)โดยอธิบาย หลักการทำงาน, ข้อดี, ข้อเสีย, และตัวอย่างการใช้งานในระบบซื้อขายออนไลน์
4. Client Component และ Server Component เกี่ยวข้องยังไงกับ 4 ประเภทของ Rendering
5. อธิบายความแตกต่างระหว่าง Page Router กับ App Router

6. กำหนดโครงสร้างไฟล์เดอร์ดังนี้

```
app/  
├── layout.tsx  
├── page.tsx  
├── about/  
│   └── page.tsx  
└── products/  
    ├── page.tsx  
    └── [id]/  
        └── page.tsx
```

จงตอบคำถามต่อไปนี้

- URL ที่สามารถเข้าถึงแต่ละ page คืออะไร
- [id] หมายถึงอะไร
- หากเข้า /products/15 ค่า id จะมีค่าเท่าใด

7. ตอบคำถามเกี่ยวกับ Environment Variables ใน Next.js ต่อไปนี้

- ตัวแปรที่ขึ้นต้นด้วย NEXT_PUBLIC_ แตกต่างจากตัวแปรทั่วไปอย่างไร
- เหตุใดจึงไม่ควรเก็บ Secret Key ไว้ในตัวแปร NEXT_PUBLIC_
- ยกตัวอย่างตัวแปรที่ควรเป็น Public และ Private อย่างละ 2 ตัว

8. พัฒนาส่วน Frontend (UI) ของระบบซื้อขายออนไลน์ โดยเลือกพัฒนา อย่างน้อย 5 features ต่อไปนี้

• ส่วนของลูกค้า

- | | |
|---|-------------------------------|
| 1. หน้าหลัก (เฉพาะส่วนโครงสร้างและเมนู) | 9. เพิ่มสินค้าเข้าตะกร้า |
| 2. สมัครสมาชิก | 10. แสดงข้อมูลในตะกร้าสินค้า |
| 3. เข้าสู่ระบบ | 11. แก้ไขข้อมูลในตะกร้าสินค้า |
| 4. แสดงโปรไฟล์/รูปโปรไฟล์ | 12. ยืนยันการสั่งซื้อ |
| 5. แก้ไขโปรไฟล์/รูปโปรไฟล์ | 13. ยืนยันชำระเงิน |
| 6. ออกจากระบบ | 14. แสดงประวัติการสั่งซื้อ |
| 7. หน้าแรก (รายการสินค้า) | |
| 8. รายละเอียดสินค้า | |

• ส่วนของพนักงาน

1. เข้าสู่ระบบ
2. โปรไฟล์พนักงาน/รูปโปรไฟล์
3. จัดการข้อมูลลูกค้า
4. จัดการข้อมูลสินค้า
5. จัดการข้อมูลประเภทสินค้า
6. จัดการข้อมูลจังหวัด
7. จัดการข้อมูลอำเภอ
8. จัดการข้อมูลตำบล
9. ตรวจสอบและอนุมัติสลิปเงิน
10. รายงานการ ซื้อขาย (รายวัน)

• ส่วนของผู้บริหาร มีทุก features เหมือนส่วนของพนักงาน ยกเว้นรายการต่อไปนี้

1. จัดการข้อมูลพนักงาน
2. รายงานการ ซื้อขาย (รายวัน/เดือน/ไตรมาส/ปี)

```
project-name/  
├── frontend/  
│   ├── nextjs/  
│   └── flutter/  
└── backend/  
    ├── express/  
    │   └── assets/  
    │       ├── customer/  
    │       ├── employee/  
    │       ├── product/  
    │       └── payment/  
    ├── flask/  
    └── mysql/  
        └── schema.sql
```

ตัวอย่างโครงสร้างไฟล์เตอร์ของ Project

THANK YOU

FOR YOUR ATTENTION



Contact Me:

 ห้อง 970

 pichai_jo@rmutto.ac.th

 busit.rmutto.ac.th

